

Take-Home Challenge — "Peblo TV Mini"

CMS upload → published catalogue → Netflix-style browse

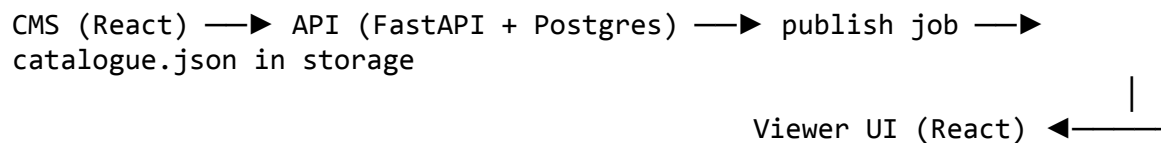
Peblo · Full-Stack Platform Engineer (Python/FastAPI + React) · v1

We grade judgment and operability, not feature count. An honest, well-reasoned 70% beats a rushed 100% — tell us in your README what you skipped and why.

1. The scenario

This is a miniature of a real Peblo product surface. Peblo TV is our streaming mode: our content team uploads a show's episodes and artwork through an **internal CMS**, the backend builds a **published catalogue file**, and a **viewer-facing UI** reads that catalogue so a child can browse rows, search, and filter — Netflix-style.

You'll build all three layers, plus the pipeline that runs them.



2. What we've given you

File	What it is
seed_shows.json	95 episode rows across 8 shows — the raw content data your CMS manages
reference.json	Allowed sections, categories, languages, artwork specs, and two conventions you must honour
assets/	A handful of sample images to upload (some are deliberately the wrong size)

Two conventions from reference.json that matter:

- **Season 0 is reserved for trailers** — it is not a normal season in the viewer UI.
- **content_group** — episodes sharing one are language variants of the *same* episode. They must collapse into **one catalogue entry listing its available languages**. This is how we ship English/Hindi.

The seed data is deliberately imperfect. We won't tell you where — finding and handling it is part of the exercise. Your validation report should surface whatever you find.

3. Part A — Backend (FastAPI + PostgreSQL)

1. **Schema + migrations:** shows → seasons → episodes, artwork records, publish runs.
2. **Artwork upload endpoint.** Three sizes per the specs in reference. `.json`: poster (2:3, ~600×900), banner (16:9, ~1280×720), thumbnail (16:9, ~640×360). Validate aspect/dimensions, enforce the 200 KB ceiling, reject with errors a non-technical editor can act on. Store behind a **storage abstraction** — we use Cloudflare R2 in production; local disk or MinIO is fine here, but swapping should be one class.
3. **CRUD** for shows/seasons/episodes with validation:
 - an episode can't be published without artwork and a duration
 - (`content_group`, `language`) must be unique
 - a published show must have a section
4. **POST /admin/catalog/publish** — build the catalogue JSON and write it to storage:
 - only published shows/episodes appear
 - `content_group` variants collapse into one entry with a `languages` list
 - grouped by section, deterministic ordering
 - the run is recorded (who, when, counts, outcome)
 - **atomic** — a reader must never see a half-written catalogue
5. **GET /catalog** — what the viewer reads (serve the published file or a fast equivalent).
6. **GET /catalog/search?q=&category=&language=§ion=** — `q` matches show title **and** episode title **and** category; all filters compose.
7. **GET /admin/validation-report** — everything currently blocking publish, grouped so an editor can fix it without asking an engineer.
8. **Roles:** editor (CRUD) vs admin (CRUD + publish) — actually enforced, not just declared.
9. **Tests** on the parts you consider risky.

4. Part B — Internal CMS (React + TypeScript)

1. Show/episode list: search, filters (section, status, language), pagination.
2. Create/edit form with **three labelled artwork upload slots** — each showing its required dimensions, a live preview, and human-readable errors. Assume the user is a content editor, not an engineer.
3. Publish page: the validation report, a publish button that's disabled *with reasons* when blocked, and run history.
4. Handle loading / empty / error / permission-denied states.
5. TanStack Query, or state your alternative and why.

Styling doesn't need to be beautiful. It needs to be usable by someone who will do this 50 times a week.

5. Part C — Viewer browse UI (React + TypeScript)

A separate route or app that reads **only the published catalogue**:

1. Netflix-style home: a **featured hero** plus horizontal rows by section, using the right artwork per surface (banner for the hero, poster for rows, thumbnail for episode lists).
2. **Search + filters** (category, language) with a sensible empty state.
3. Show detail: synopsis, seasons and episodes, and the language options for a grouped episode. Trailers (Season 0) shouldn't appear as a normal season.
4. It should stay pleasant when images are slow — your call how.

6. Part D — Pipeline & operability

1. docker-compose up brings up API, database, and both UIs, seeded and working.
2. A **GitHub Actions** workflow: lint, tests, build images. The deploy step should be written and explained — it doesn't need a real cloud to deploy to.
3. .env.example covering every variable, plus a paragraph on how you'd manage these secrets in production.
4. A health endpoint, and one thing you'd alert on, with your reasoning.

7. Part E — Written (max 1 page, in the README)

1. How you made publishing atomic — and what happens if the process dies mid-publish.
2. Your storage abstraction: what changes to move from local disk to Cloudflare R2?
3. Search: how did you implement it, at what catalogue size does it stop working, and what would you do next?
4. Why serve a pre-published catalogue file at all instead of querying the database per request? Where does that choice bite you?
5. What you left out and why. Which AI tools you used, and where you accepted or rejected their output. (Using them is fine — we want your judgment, not your typing speed.)

8. Optional stretch

Only if time genuinely remains, and say so explicitly: versioned catalogue with rollback to a previous run · a publish dry-run showing a diff · an audit log of who changed what.

9. What to send us

A Git repo (public link or shared access) containing your code and a README with: how to run it, your decisions and trade-offs, and roughly how long you spent on each part.

Also, send a screen recording of how the flow works.

10. How we'll score it (100 points)

Area	Points
Upload & validation — three sizes genuinely enforced, storage abstracted, editor-readable errors	15
Publish job — atomic, recorded, idempotent, language grouping correct	20
API design & auth — sensible resources, roles enforced, honest errors, filters compose	15
Data modelling — schema fits the queries, indexes justified, clean migrations	10
CMS usability — an editor could use it unaided, all states handled	15
Viewer UI — hero/rows/detail correct, right artwork per surface, search & filters, empty states	10
Pipeline & operability — compose works first try, CI meaningful, secrets & alerting reasoned	10
Written reasoning — real trade-offs	5

Things that will count against you: publishing by overwriting the live file · artwork accepted at any size · roles declared but never enforced · client-side-only validation · search done in the browser over the whole catalogue with no comment on scale · the viewer UI calling admin endpoints · docker-compose up not working.

11. Questions

If something's ambiguous, make a decision, note it in the README, and move on — that's what we'd want on the job too. If you're genuinely blocked, email [contact] and we'll answer within a few hours.

All Links are provided here -

■ thumb_tiny.jpg

■ thumb_good.jpg

[seed_shows.json](#)

[reference.json](#)

📄 poster_wrong_ratio.jpg

📄 poster_good.jpg

[CHALLENGE Peblo TV Mini.md](#)

📄 CHALLENGE_Peblo_TV_Mini.docx

📄 banner_too_big.png

📄 banner_good.jpg